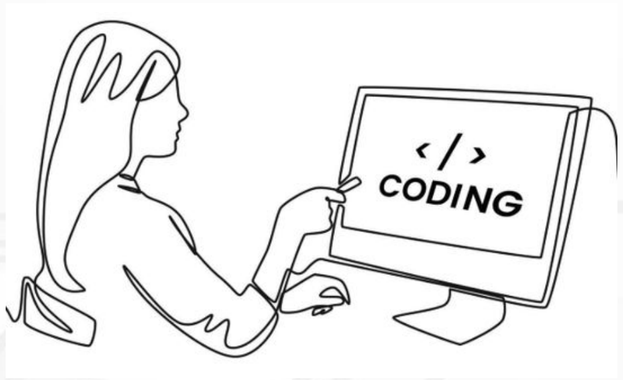
FADENA
FACULTAD DE DEFENSA NACIONAL

Lenguajes de Programación

Algoritmos básicos y arreglos

Docentes:

- Ing. Sebastián Balestrini
- Esp. Ing. Gustavo Di Stefano



Objetivos

- Punteros y sus operadores
- Vectores y matrices
- Recorridos con for
- Búsqueda lineal
- Ordenamiento burbuja básico



Herramientas

OnlineGDB es un entorno de desarrollo integrado (IDE) en la nube que combina edición de código, compilación y ejecución de programas en C/C++ con un solo clic:

https://www.onlinegdb.com/

,

Repaso de la unidad anterior

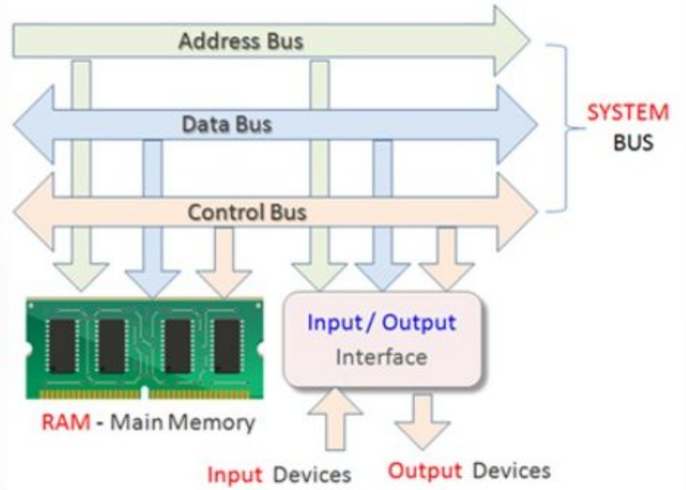
- Como declarar constantes
  - **const** y **#define**
- Como declarar variables
- Tipos de datos: char, int, float, double
- Estructuras de control
  - Selectivas: **if** y **case**
  - Repetitivas: **for**, **while**, **do while**



Anatomía básica de una computadora

- 🧠 CPU (Unidad Central de Procesamiento):
    - Ejecuta las instrucciones de los programas.
- 📦 Memoria RAM (Memoria de Trabajo)
    - Almacena datos e instrucciones en uso. Volátil (se borra al apagar).
- 💾 Almacenamiento (Disco Duro/SSD)
    - Guarda el sistema operativo, programas y archivos de forma persistente.
- 🔌 Periféricos y Microcontroladores
    - Gestionan dispositivos de entrada/salida (teclado, puertos, sensores).
- 🔄 Buses del Sistema
    - Son las "autopistas" que conectan todos los componentes y permiten la comunicación.







FADENA
FACULTAD DE DEFENSA NACIONAL

La memoria RAM

- Podemos imaginar la RAM como una tabla con filas numeradas.
- Cada fila tiene:
  - Una dirección (número que identifica el casillero).
  - Un contenido (el valor que guarda).
- Una variable ocupa un casillero.
- El nombre de la variable es un “alias” para recordar esa dirección.

Estructura de la memoria RAM

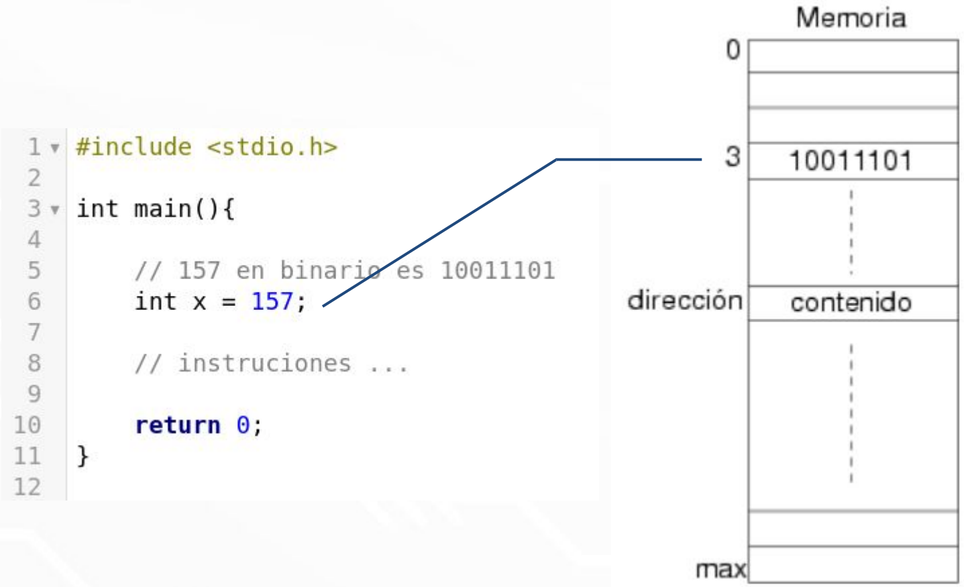
```c
#include <stdio.h>

int main(){

    // 157 en binario es 10011101
    int x = 157;

    // instruciones ...

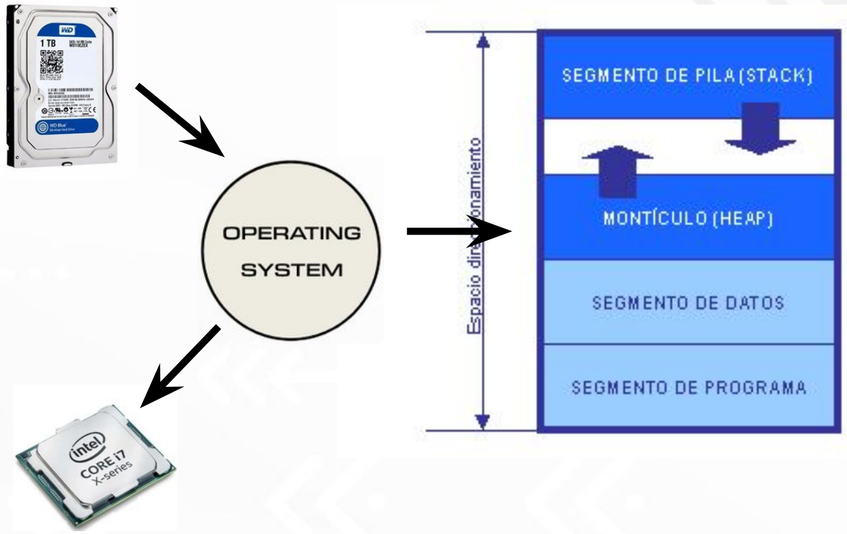
    return 0;
}
```





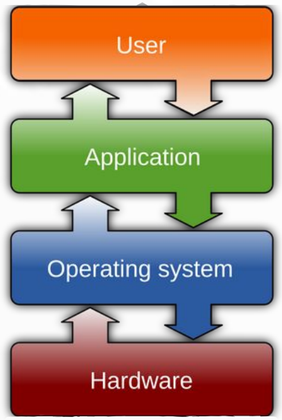
FADENA
FACULTAD DE DEFENSA NACIONAL

El viaje de un programa



¿Qué pasa cuando ejecutamos un programa?

- El sistema operativo carga el programa en memoria.
- La memoria se divide en secciones:
  - Código → instrucciones del programa.
  - Datos → globales y estáticos.
  - Pila → variables locales y control de funciones.
  - Heap → memoria dinámica bajo demanda.
- La CPU empieza a leer instrucciones y ejecutarlas en ciclos.



FADENA
FACULTAD DE DEFENSA NACIONAL

Punteros

¿ Qué es un puntero ?

- Un puntero es una variable especial que ***guarda direcciones de memoria***
- Se dice que "**apunta a**" dirección de memoria, que contiene un valor con un tipo específico de dato, definido previamente.

¿ Cómo se define ?

- tipo ***nombrePuntero;**
- Se utilizan los operadores de referencia (**&**) y desreferencia (***)**
- Operador de referencia (**&**): obtiene la direccion de memoria de una variable
- Operador de desreferencia (***)**: acceder al valor que hay en esa dirección.

Ejemplo

```c
#include <stdio.h>

int main() {
    int x = 42;        // una variable normal
    int *p = &x;       // p guarda la dirección de x (uso &)

    printf("Valor de x: %d\n", x);
    printf("Direccion de x (&x): %p\n", &x);
    printf("Direccion guardada en p: %p\n", p);

    // Uso * para acceder al valor desde el puntero
    printf("Valor al que apunta p (*p): %d\n", *p);

    // Modifico el valor de x usando el puntero
    *p = 99;   // aquí uso * para desreferenciar y asignar un nuevo valor

    printf("Nuevo valor de x: %d\n", x);
    printf("Valor al que apunta p (*p): %d\n", *p);

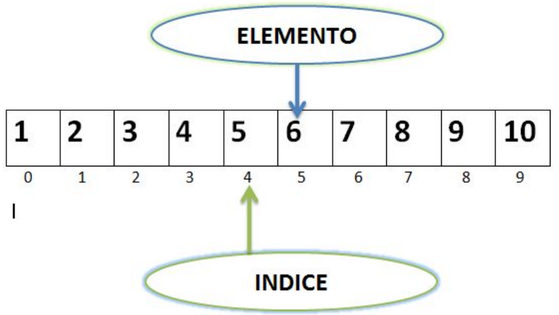
    return 0;
}
```

```
Valor de x: 42
Direccion de x (&x): 0x7ffee00f4c1c
Direccion guardada en p: 0x7ffee00f4c1c
Valor al que apunta p (*p): 42
Nuevo valor de x: 99
Valor al que apunta p (*p): 99
```

Vectores

- **Variables**: ocupan un casillero en memoria.
  - `int x = 5;`
- **Punteros**: guardan direcciones de memoria.
  - `int *px = &x;`
- **Vectores**: varios casilleros contiguos en memoria.
  - `int vector[10];`
- El índice (n) del vector comienza desde 0 a (n-1)

	Dirección	Memoria
	0	
x	1	5
	2	
	…	
p	8	1



FADENA
FACULTAD DE DEFENSA NACIONAL

Arreglos

- **Arreglos unidimensional** → vector

```
int vector[5];
```

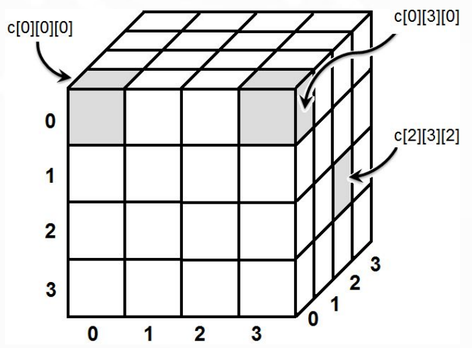
- **Arreglos bidimensional** → matriz

```
int matriz[3][5];
```

- **Arreglos multidimensional** → mayor a 2 dimensiones

```
int tridimensional[4][4][4];
```





FADENA
FACULTAD DE DEFENSA NACIONAL

Recorrido con for

Recorrido de un vector

```c
#include <stdio.h>

int main() {
    int numeros[5] = {10, 20, 30, 40, 50}; // Vector de 5 elementos

    // Recorrido del vector usando un for
    for (int i = 0; i < 5; i++) {
        printf("Elemento en la posición %d: %d\n", i, numeros[i]);
    }

    return 0;
}
```

```
Elemento en la posición 0: 10
Elemento en la posición 1: 20
Elemento en la posición 2: 30
Elemento en la posición 3: 40
Elemento en la posición 4: 50
```

Recorrido de una matriz

```c
#include <stdio.h>

int main() {
    int matriz[3][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };

    // Recorrido de la matriz
    for (int fila = 0; fila < 3; fila++) {
        for (int col = 0; col < 3; col++) {
            printf("Elemento [%d][%d]: %d\n", fila, col, matriz[fila][col]);
        }
    }

    return 0;
}
```

```
Elemento [0][0]: 1
Elemento [0][1]: 2
Elemento [0][2]: 3
Elemento [1][0]: 4
Elemento [1][1]: 5
Elemento [1][2]: 6
Elemento [2][0]: 7
Elemento [2][1]: 8
Elemento [2][2]: 9
```

Operaciones con vectores

Ordenamiento Bubble Sort

- Bubble Sort es un método simple para ordenar vectores.
- Compara elementos adyacentes y los intercambia si están en el orden incorrecto.
- Se repite hasta que todo el vector queda ordenado.
- Cada pasada asegura que el mayor elemento "burbujee" hasta el final.

Ordenamiento

```c
#include <stdio.h>

int main() {
    int numeros[6] = {30, 10, 50, 20, 40, 15};
    int n = 6;

    // -------- Ordenamiento burbuja --------
    for (int i = 0; i < n-1; i++) {
        for (int j = 0; j < n-i-1; j++) {
            if (numeros[j] > numeros[j+1]) {
                // Intercambiamos
                int temp = numeros[j];
                numeros[j] = numeros[j+1];
                numeros[j+1] = temp;
            }
        }
    }

    // Mostrar vector ordenado
    printf("Vector ordenado: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", numeros[i]);
    }
    printf("\n");

    return 0;
}
```

Operaciones con vectores

Búsqueda Lineal en un Vector

- La búsqueda lineal consiste en revisar cada elemento del vector de manera secuencial hasta encontrar el valor buscado.
- Es simple y no requiere que el vector esté ordenado, aunque funciona igual en vectores ordenados.

Recorrido de una matriz

```c
#include <stdio.h>

int main() {
    int numeros[6] = {10, 15, 20, 30, 40, 50}; // Vector ya ordenado
    int n = 6;
    int buscar = 20;
    int encontrado = -1;  // -1 indica que no se encontró

    // -------- Búsqueda lineal --------
    for (int i = 0; i < n; i++) {
        if (numeros[i] == buscar) {
            encontrado = i;
            break;  // Detenemos la búsqueda
        }
    }

    if (encontrado != -1) {
        printf("El número %d se encuentra en la posición %d.\n", buscar, encontrado);
    } else {
        printf("El número %d no se encuentra en el vector.\n", buscar);
    }

    return 0;
}

```

Que aprendimos hasta acá ?

- Como funciona una computadora
- Como funciona una memoria
- Como arranca un programa
- Punteros
- Vectores y arreglos
- Operaciones con vectores: Búsqueda y Ordenamiento



Próximo paso

Funciones y modularización

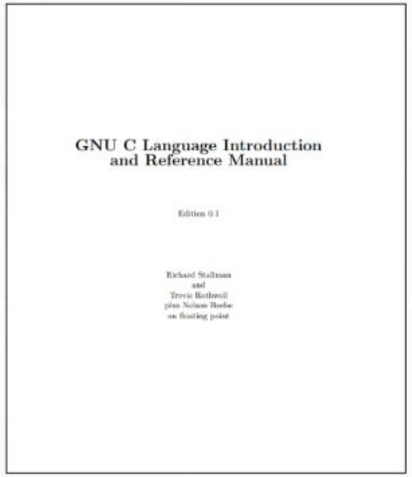
Bibliografía

- El lenguaje C: Manual de Referencia (Balestrini - Di Stefano)
- Programación en C para principiantes (Urrutia - Landa)
- Introducción a la programación en C (Marzal - Gracia)
- GNU C Language Introduction and Reference Manual (Richard Stallman)









FADENA
FACULTAD DE DEFENSA NACIONAL

¿Preguntas o dudas?

